Pingfin 2026

Projectverslag — Bankteam

International Week 2026 — Professional Development

Team	Team 15
Teamleden	Azzouzi Fadil, Hammouche Ayoub, Kallouch Ilyas
BIC	BOFABE3X / FXBBBEBB
Banktype	Regular Bank (OB & BB)
Coaches	Sam Van Buggenhout & Jens Baetens
Lokaal	HER 3 — 4306
Academiejaar	2025–2026
Datum	30 april 2026
Instelling	Odisee — De Co-Hogeschool
Opleiding	Bachelor Toegepaste Informatica

Inhoudstafel

1. Inleiding ... 3
- 1.1 Beschrijving van het project ... 3
- 1.2 Doel van het project ... 3
- 1.3 Schema van de opbouw van het project ... 4

2. Communicatie tussen GUI en API ... 5
- 2.1 Authenticatie ... 5
- 2.2 Standaard responsformaat ... 5
- 2.3 Databankstructuur ... 6

3. Documentatie van de API ... 7
- 3.1 Authenticatie ... 7
- 3.2 Foutcodes ... 7
- 3.3 Rate limiting ... 8
- 3.4 Publieke endpoints ... 8
- 3.5 OB-workflow endpoints ... 9
- 3.6 BB-workflow endpoints ... 10
- 3.7 Overige endpoints ... 11
- 3.8 Admin-endpoints ... 11
- 3.9 Voorbeeldgebruik met Postman ... 12

4. Flow van een Payment Order ... 13
- 4.1 Beschrijving van het proces ... 13
- 4.2 Use Cases ... 14
- 4.3 Uitzonderingen ... 15

5. Plan van aanpak ... 16
- 5.1 Werkverdeling en planning ... 16
- 5.2 Werkverdeling ... 17
- 5.3 Gebruikte tools en methodologie ... 17
- 5.4 Problemen en oplossingen ... 18

6. Handleiding voor het gebruik van de GUI ... 19
- 6.1 Overzicht ... 19
- 6.2 Inloggen ... 19
- 6.3 Admin-interface ... 20
- 6.4 User-interface ... 22

7. Reflectie ... 23
- 7.1 Terugblik op het project ... 23
- 7.2 Geleerde lessen ... 23
- 7.3 Verbeterpunten voor toekomstige projecten ... 24

8. Bijlagen ... 25

1. Inleiding

1.1 Beschrijving van het project

Pingfin is een projectweek georganiseerd door Odisee binnen het vak Professional Development (2STP) van de Bachelor Toegepaste Informatica. Gedurende vier dagen (27–30 april 2026) werkt elk team aan een banktoepassing die betalingstransacties verwerkt binnen een gesimuleerd SEPA-systeem (Single Euro Payments Area).

Team 15 speelt de rol van gewone bank (OB en BB) met twee BIC-codes: BOFABE3X en FXBBBEBB. Elke bank stuurt betalingen (Payment Orders) via een centrale Clearing Bank naar andere teams, en verwerkt omgekeerd inkomende betalingen. De volledige communicatie verloopt via REST APIs.

1.2 Doel van het project

Het doel van het project is het bouwen van een werkend interbankenbetalingssysteem dat:

- Payment Orders kan aanmaken, valideren en verzenden naar een Clearing Bank (OB-rol)
- Inkomende betalingen van andere banken kan ontvangen en verwerken (BB-rol)
- Rekeningsaldo's correct bijhoudt via een relationele database
- Een GUI biedt voor zowel bankmedewerkers (admin) als rekeninghouders (user)
- Volledig gecontaineriseerd draait via Docker Compose

1.3 Schema van de opbouw van het project

De applicatie bestaat uit drie lagen:

Laag	Technologie	Beschrijving
GUI (frontend)	Vanilla HTML/CSS/JS	Single-page applicatie met admin- en gebruikersinterface
API (backend)	Node.js + Express.js	REST API met OB/BB logica, validatie en CB-integratie
Database	MySQL / MariaDB	Opslag van accounts, POs, ACKs, transacties en logs

De drie lagen communiceren als volgt: de GUI roept de API aan via HTTP (Bearer JWT). De API verwerkt de businesslogica en schrijft naar de database via een connection pool (mysql2). De API communiceert ook met de externe Clearing Bank via HTTP (Bearer token, 4u geldig).

2. Communicatie tussen GUI en API

2.1 Authenticatie

De GUI communiceert met de API via HTTP-requests. Elke request naar een beveiligd endpoint bevat een JWT Bearer token in de Authorization-header:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Het token wordt na een succesvolle login (POST /api/auth/login) teruggegeven en opgeslagen in localStorage. Het token is 4 uur geldig. Bij een 401-respons logt de GUI automatisch uit.

2.2 Standaard responsformaat

Elk API-endpoint geeft een gestandaardiseerd JSON-object terug:

```
{
  "ok":      true | false,
  "status":  200 | 400 | 401 | 403 | 404 | 500,
  "code":    "2000",
  "message": "OK",
  "data":    { ... } | [ ... ] | null
}
```

2.3 Databankstructuur

De database pingfin bevat de volgende tabellen:

Tabel	Beschrijving
users	Gebruikers van de GUI/API (username, bcrypt hash, rol: admin of user)
accounts	Bankrekeningen (IBAN als primaire sleutel, saldo met CHECK balance >= 0)
po_new	Nieuw aangemaakte POs die nog niet verzonden zijn (status: pending/processed/failed)
po_out	POs verzonden naar de CB, wachten op ACK (sent_to_cb flag)
po_in	POs ontvangen van de CB als BB (status: received/processed/failed)
ack_in	ACKs ontvangen van de CB als OB (received_at timestamp)
ack_out	ACKs verzonden naar de CB als BB (sent_to_cb flag)
transactions	Alle debet- en credittransacties op rekeningen (isvalid, iscomplete)
logs	Alle systeemgebeurtenissen met volledig PO-context (alle 14 PO-velden)

Alle PO-tabellen (po_new, po_out, po_in, ack_in, ack_out) delen dezelfde 14 velden conform de manual: po_id, po_amount, po_message, po_datetime, ob_id, oa_id, ob_code, ob_datetime, cb_code, cb_datetime, bb_id, ba_id, bb_code, bb_datetime.

3. Documentatie van de API

3.1 Authenticatie

De API gebruikt JWT (JSON Web Token) authenticatie met het HS256-algoritme. Tokens zijn 4 uur geldig en bevatten de payload { id, username, role }. Er zijn twee rollen: admin (volledige toegang) en user (operationele endpoints).

3.2 Foutcodes

Onderstaande tabel geeft een overzicht van alle foutcodes die door de API gebruikt worden:

Code	Type	Betekenis
2000	Succes	OK — operatie geslaagd
3001	OB fout	Originerende rekening (OA) niet gevonden
3002	OB fout	Onvoldoende saldo op OA
3003	OB fout	Ongeldig PO-formaat (validatie mislukt)
3004	OB fout	Bedrag overschrijdt limiet van €500
3005	OB fout	Bedrag moet positief zijn
3006	OB fout	Dubbel po_id (reeds aanwezig)
3007	OB fout	Interne betaling (OB == BB) niet toegestaan
4001	CB fout	Interne transactie — mag niet naar CB gestuurd worden
4002	CB fout	Bedrag overschrijdt €500 limiet
4003	CB fout	Bedrag mag niet negatief zijn
4004	CB fout	Begunstigde bank (BB) bestaat niet bij CB
4005	CB fout	PO reeds ontvangen door CB
5001	BB fout	Begunstigde rekening (BA) niet gevonden
5002	BB fout	Ongeldig PO-formaat aan BB-zijde
5003	BB fout	Dubbel PO ontvangen
9001	Netwerk	CB API niet bereikbaar of timeout
9002	Netwerk	Ongeldige respons van CB
9003	Netwerk	Authenticatiefout (ongeldig token of credentials)

3.3 Rate limiting

Alle /api/* endpoints zijn beperkt tot 300 requests per IP per 60 seconden. Bij overschrijding: HTTP 429 Too Many Requests.

3.4 Publieke endpoints (geen authenticatie vereist)

De volgende endpoints zijn zonder token toegankelijk en worden gebruikt door andere teams om bankinfo op te vragen:

Methode	Endpoint	Beschrijving
`GET`	`/api/help`	Overzicht van alle beschikbare endpoints

Methode	Endpoint	Beschrijving
GET	/api/info	Identiteitsinformatie van de bank (BIC, naam, teamleden)
GET	/api/accounts	Lijst van alle rekeningen met saldo's
GET	/api/banks	Lijst van deelnemende banken (proxied via CB)
GET	/api/errorcodes	Overzicht van alle foutcodes (lokaal + CB)
POST	/api/auth/login	JWT token ophalen (body: { username, password })
GET	/health	Server health check

3.5 OB-workflow endpoints (Bearer token vereist)

De Originating Bank-workflow verloopt in vier stappen: PO aanmaken → valideren → verzenden naar CB → ACK ophalen.

Methode	Endpoint	Beschrijving
GET	/api/po_new_generate	Random POs genereren (?count=N&errors=true/false)
POST	/api/po_new_add	POs toevoegen aan PO_NEW (body: { data: [PO,...] })
GET	/api/po_new_process	PO_NEW valideren → geldige POs naar PO_OUT verplaatsen
GET	/api/po_out_send	PO_OUT verzenden naar CB via POST /po_in
GET	/api/ack_pull	ACKs ophalen van CB → ACK_IN, OA-saldo bijwerken

Validatieregels bij po_new_process

- IBAN-formaat en mod-97 checksum (OA en BA)
- BIC-formaat — 8 of 11 tekens (OB en BB)
- Bedrag > 0 en ≤ €500, maximaal 2 decimalen
- Datetime-formaat: YYYY-MM-DD HH:MM:SS
- OA-rekening bestaat in de eigen bank
- Voldoende saldo op OA
- OB ≠ BB (geen interne betalingen via CB)

3.6 BB-workflow endpoints (Bearer token vereist)

De Beneficiary Bank-workflow verloopt in drie stappen: POs ophalen van CB → verwerken → ACK terugsturen.

Methode	Endpoint	Beschrijving
GET	/api/po_pull	POs ophalen van CB → opslaan in PO_IN
GET	/api/po_in_process	PO_IN verwerken: BA crediteren, ACK aanmaken in ACK_OUT
GET	/api/ack_out_send	ACK_OUT verzenden naar CB via POST /ack_in

3.7 Overige endpoints

De volgende endpoints combineren meerdere stappen of geven data-overzichten terug:

Methode	Endpoint	Beschrijving
GET	/api/cycle	Volledige OB+BB cycle in één request (6 stappen)
POST	/api/payments	Betaling aanmaken en verzenden in één stap
GET	/api/po_new|po_out|po_in	Tabelinhoud opvragen (max 200 records)
GET	/api/ack_in|ack_out	Tabelinhoud opvragen (max 200 records)
GET	/api/transactions	Transactiehistorie (?account=IBAN voor filtering)
GET	/api/log	Systeemlogboek (?limit=N, max 500)
GET	/api/stats	Dashboard — live tellingen en totaal saldo

3.8 Admin-endpoints (rol admin vereist)

De volgende endpoints zijn uitsluitend beschikbaar voor gebruikers met de rol admin:

Methode	Endpoint	Beschrijving
POST	/api/accounts	Nieuwe rekening aanmaken (body: { id, balance })
PUT	/api/accounts/:iban/balance	Saldo aanpassen (body: { amount })
DELETE	/api/accounts/:iban	Rekening verwijderen (saldo = 0, geen openstaande POs)
GET	/api/users	Lijst van alle gebruikers
POST	/api/users	Gebruiker aanmaken (body: { username, password, role })
DELETE	/api/users/:id	Gebruiker verwijderen (niet zichzelf)

3.9 Voorbeeldgebruik met Postman

Hieronder de stap-voor-stap Postman-simulatie van een volledige betalingscyclus.

Stap 1 — Token ophalen

```
POST http://localhost:3000/api/auth/login
Content-Type: application/json

{ "username": "admin", "password": "admin123" }
```

Response: { "ok": true, "data": { "token": "eyJ..." } }

Stap 2 — POs genereren

```
GET http://localhost:3000/api/po_new_generate?count=5&errors=false
Authorization: Bearer eyJ...
```

Response: array van 5 PO-objecten (nog niet in de database).

Stap 3 — POs toevoegen aan PO_NEW

```
POST http://localhost:3000/api/po_new_add
Authorization: Bearer eyJ...
Content-Type: application/json
```

```
{ "data": [ { "po_id": "BOFABE3X_1714294245000-0-a1b2",
              "po_amount": 250.50,
              "po_message": "Betaling voor diensten",
              "po_datetime": "2026-04-28 10:30:45",
              "ob_id": "BOFABE3X",
              "oa_id": "BE51233543472462",
              "bb_id": "GKCCBEBB",
              "ba_id": "BE72599632030243" } ] }
```


Stap 4 — Valideren en verzenden

```
GET http://localhost:3000/api/po_new_process
GET http://localhost:3000/api/po_out_send
```

Of alles in één stap via de cycle-endpoint:

```
GET http://localhost:3000/api/cycle
```

Stap 5 — ACK ophalen

```
GET http://localhost:3000/api/ack_pull
```

Bij een succesvolle ACK (cb_code=2000, bb_code=2000) wordt het saldo van de OA automatisch verlaagd en een transactierecord aangemaakt.

4. Flow van een Payment Order

4.1 Beschrijving van het proces

Elke betaling (Payment Order of PO) loopt via drie partijen: de Originating Bank (OB), de Clearing Bank (CB) en de Beneficiary Bank (BB). Het PO-bericht bevat 14 velden die stap voor stap door elke partij worden ingevuld.

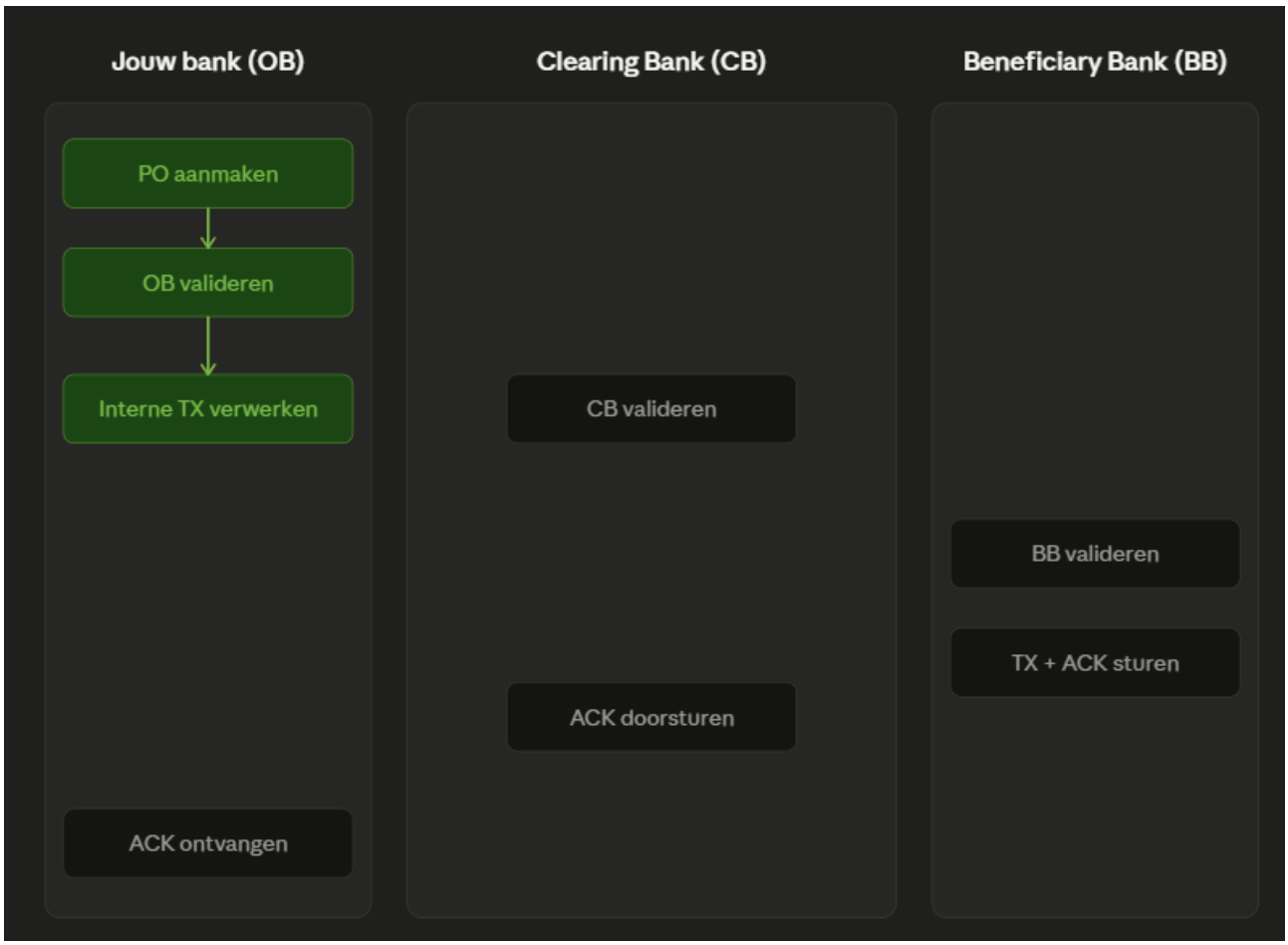
Stap	Actor	Actie	API-call
1	OB	PO aanmaken en toevoegen aan PO_NEW	`POST /api/po_new_add`
2	OB	PO_NEW valideren → PO_OUT	`GET /api/po_new_process`
3	OB	PO_OUT verzenden naar CB	`GET /api/po_out_send → CB POST /po_in`
4	CB	PO verwerken, cb_code en cb_datetime invullen	(CB intern)
5	BB	PO ophalen van CB outbox	`GET /api/po_pull → CB GET /po_out`
6	BB	BA crediteren, ACK aanmaken in ACK_OUT	`GET /api/po_in_process`
7	BB	ACK terugsturen naar CB	`GET /api/ack_out_send → CB POST /ack_in`
8	CB	ACK doorsturen naar OB outbox	(CB intern)
9	OB	ACK ophalen, OA debiteren	`GET /api/ack_pull → CB GET /ack_out`

4.2 Use Cases

Vijf use cases beschrijven alle mogelijke uitkomsten van een betalingspoging:

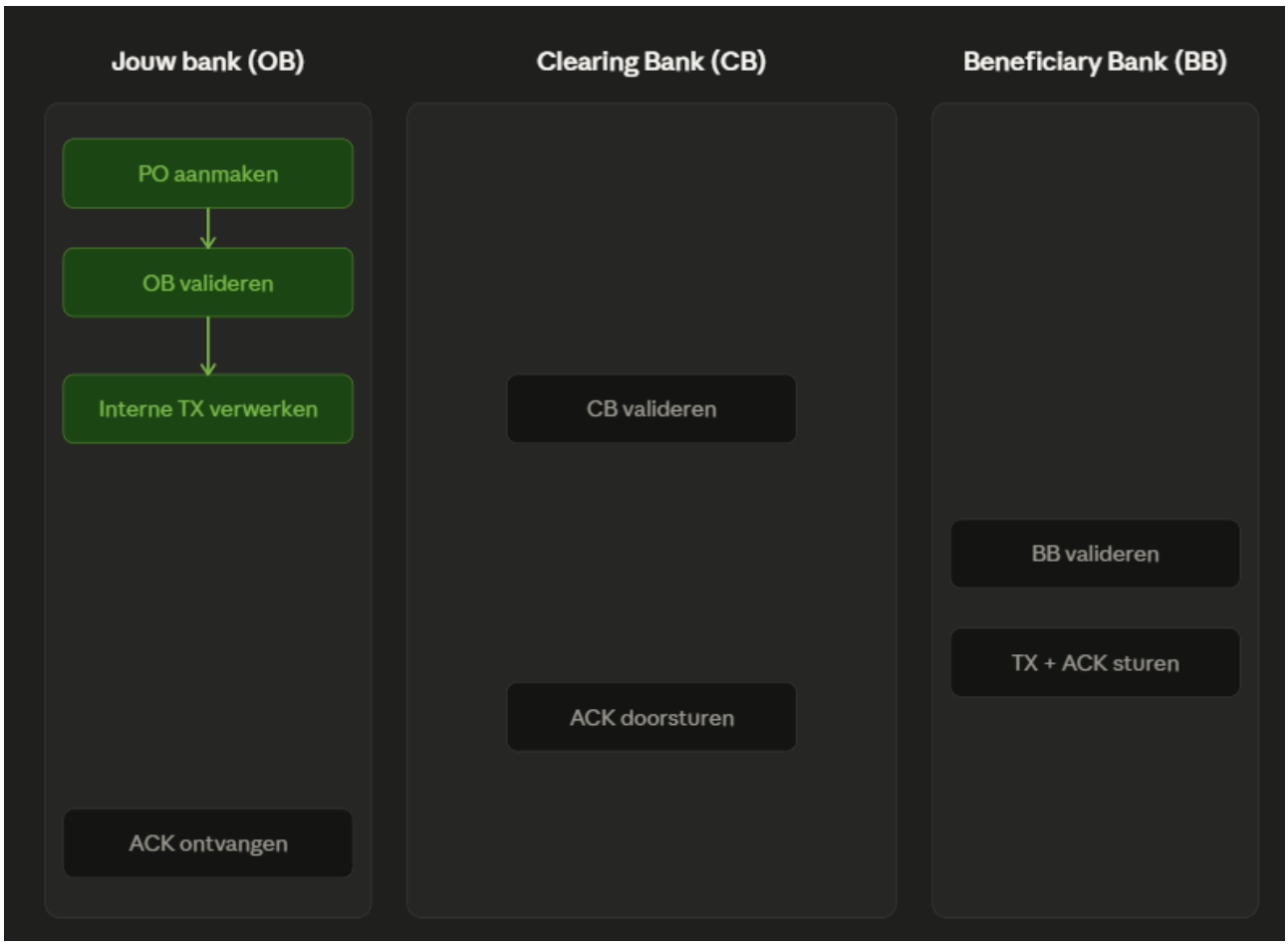
Use Case 1 — OB-validatie mislukt

De OB kent de betaalrekening (OA) niet, het bedrag is ongeldig of een veld ontbreekt. De PO verlaat de OB nooit. Het ob_code-veld wordt ingevuld met de foutcode (bv. 3001, 3002, 3003), ob_datetime krijgt het tijdstip van de actie. Geen verdere verwerking, geen CB- of BB-contact.



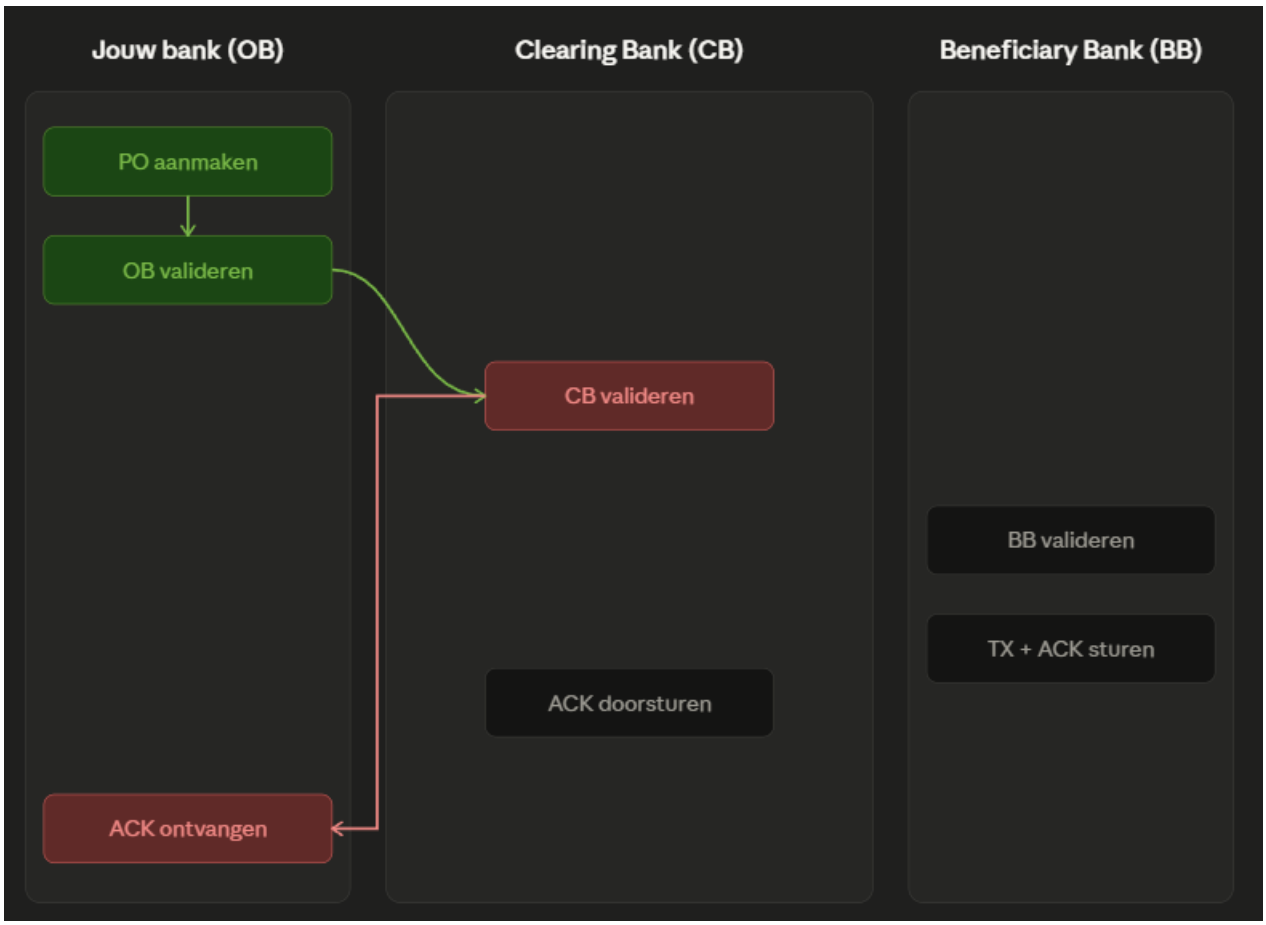
Use Case 2 — Interne betaling

OA en BA zijn rekeningen van dezelfde bank (OB == BB). De OB verwerkt intern zonder CB of BB. ob_code = 3007, ob_datetime ingevuld. De betaling wordt niet naar de CB gestuurd.



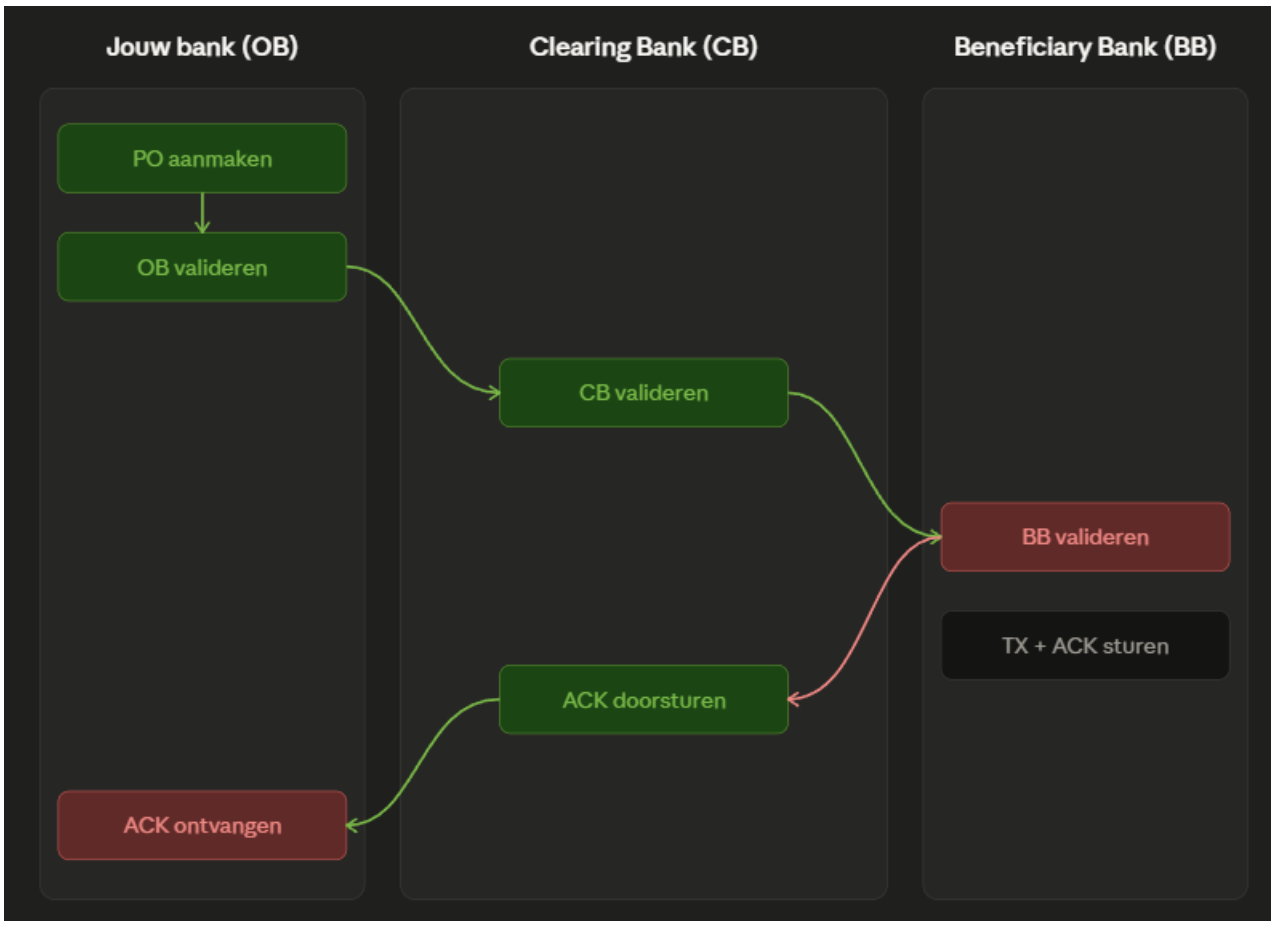
Use Case 3 — CB-validatie mislukt

OB-validatie slaagt, maar de CB kent de bestemmingsbank niet of het bedrag overschrijdt de limiet. De CB stuurt onmiddellijk een fout-ACK terug. cb_code = foutcode (bv. 4004), cb_datetime ingevuld. BB wordt niet gecontacteerd.



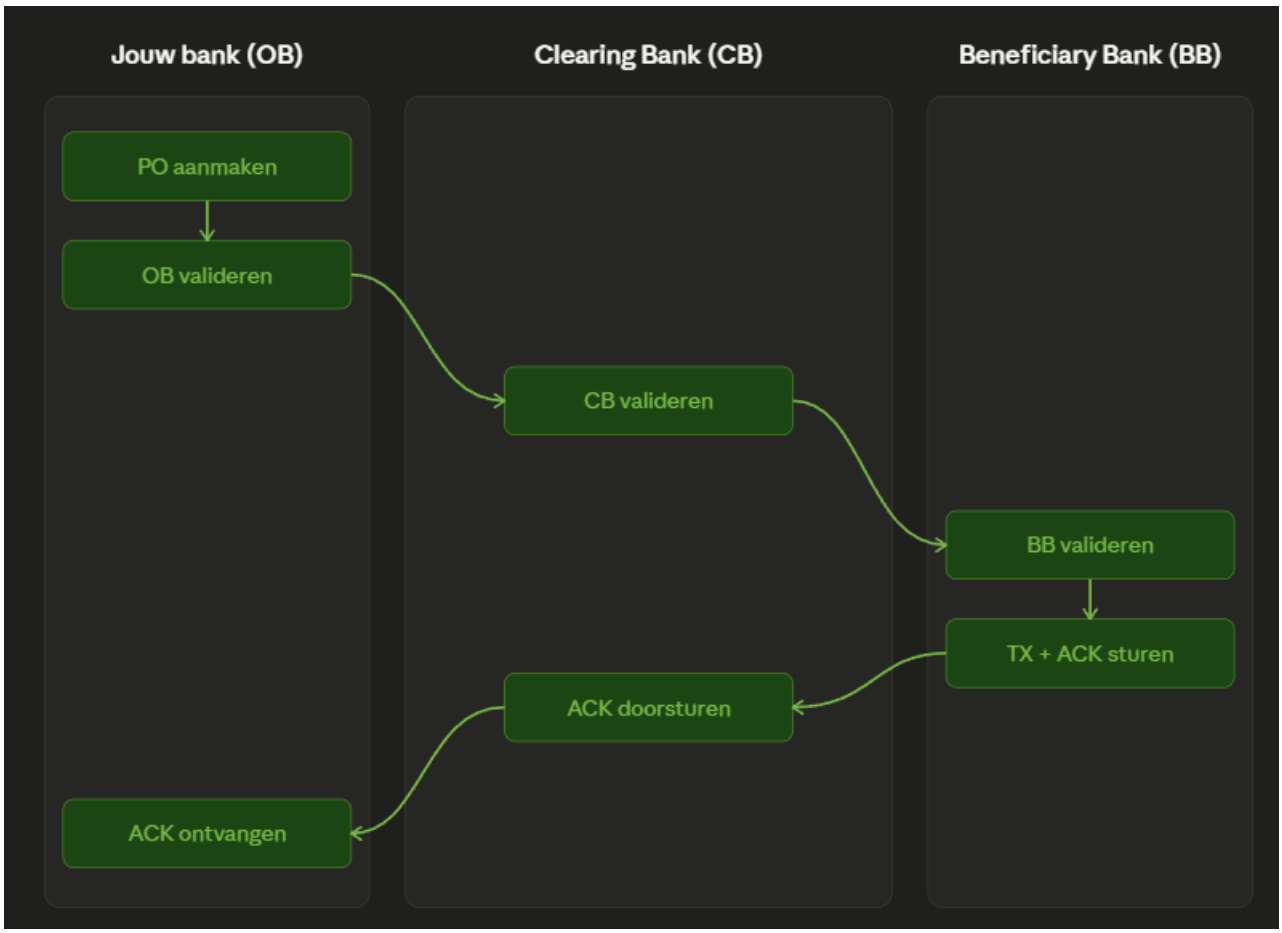
Use Case 4 — BB-validatie mislukt

OB- en CB-validaties slagen. De BB kent de bestemmingsrekening (ba_id) niet. Een fout-ACK reist via CB terug naar de OB. bb_code = foutcode (bv. 5001), bb_datetime ingevuld. De BA wordt niet gecrediteerd.



Use Case 5 — Alles geslaagd (happy path)

Alle validaties slagen. ob_code + ob_datetime (OB), cb_code + cb_datetime (CB), bb_code + bb_datetime (BB) worden allen ingevuld met code 2000. BA gecrediteerd, OA gedebiteerd, transactierecords aangemaakt.

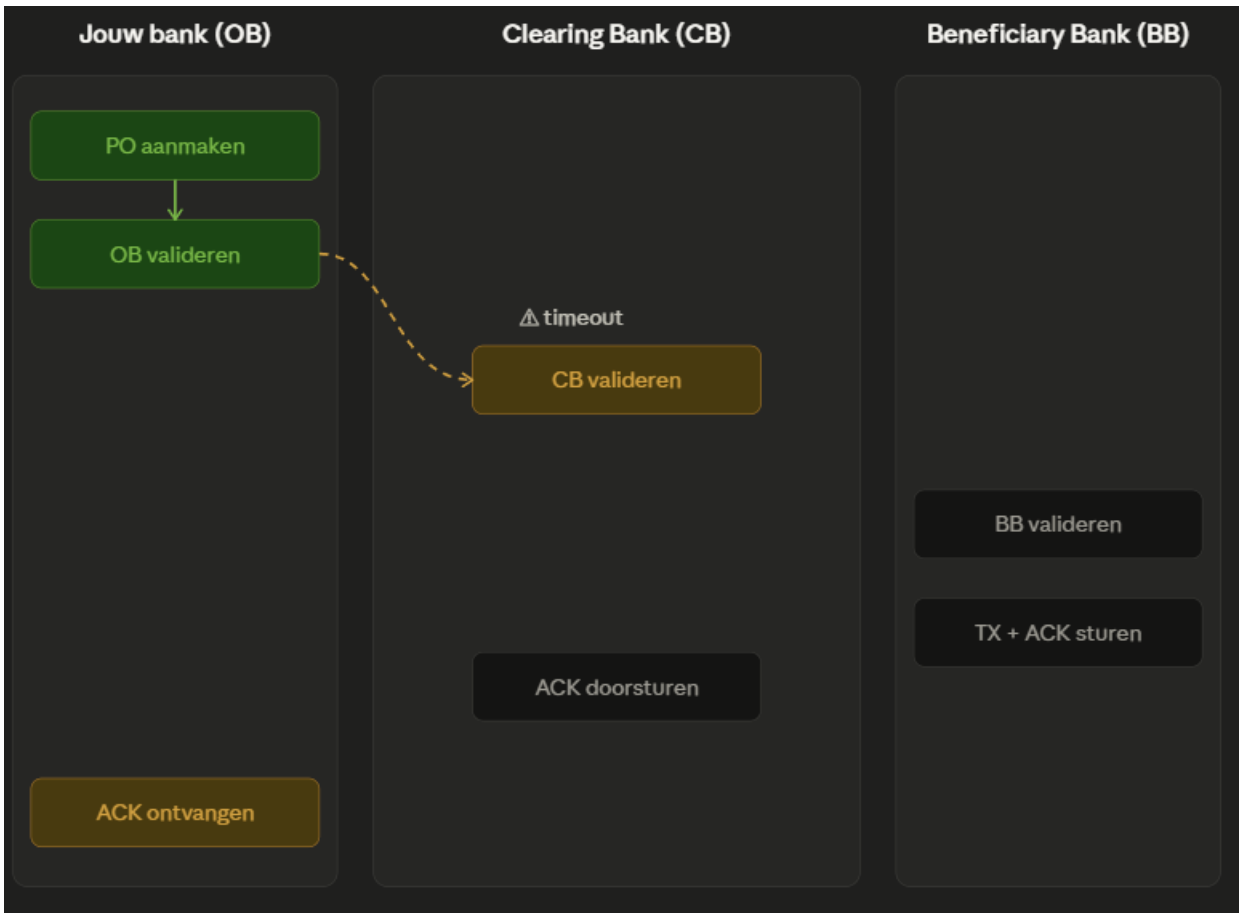


4.3 Uitzonderingen

Naast de vijf use cases zijn er twee uitzonderingssituaties die de normale flow onderbreken:

Uitzondering 1 — CB API niet bereikbaar

De OB verstuurt de PO maar de CB API geeft geen antwoord. Na timeout (15s) wordt code 9001 gelogd en de PO als "pending" gemarkeerd (sent_to_cb = 0 blijft). De achtergrondpoller (elke 30 seconden) herprobeert automatisch niet-verzonden POs.


Jouw bank (OB)
Clearing Bank (CB)
Beneficiary Bank (BB)
PO aanmaken
OB valideren
⚠ timeout
CB valideren
BB valideren
TX + ACK sturen
ACK doorsturen
ACK ontvangen

5. Plan van aanpak

5.1 Werkverdeling en planning

De projectweek was opgedeeld in vier werkdagen. Hieronder een overzicht van de taken per dag:

Dag	Activiteiten	Deliverables
Maandag 27/4	Test Docker/Node.js/Express.js, opzetten project, analyse berichtenschema, databankontwerp, Postman-simulatie met stevenop.be CB API	Repository, Trello-bord, databank SQL, Postman-screenshots
Dinsdag 28/4	Volledige API implementeren (OB + BB endpoints, validatie, CB-client, logger), API testen, Docker Compose werkend, achtergrondpoller	Werkende API op Docker
Woensdag 29/4	API testen met andere teams (cross-team), GUI afwerken, API-documentatie schrijven, deployment klaar voor network team	API-documentatie, GitHub repo, mail network team
Donderdag 30/4	Eindpresentatie voorbereiden, verslag afwerken, laatste tests, alles uploaden op Toledo	Presentatie, volledig verslag, zip-upload

5.2 Werkverdeling

De taken werden als volgt verdeeld over de teamleden:

Teamlid	Verantwoordelijkheden
Azzouzi Fadil	API-backend (OB/BB processors, CB-client, validators, poller), projectcoördinatie, verslag, deployment guidelines
Hammouche Ayoub	GUI ontwikkeling (admin + user interface), Docker setup, API-documentatie
Kallouch Ilyas	Database-ontwerp, testing (Postman), berichtenschema-analyse, presentatie voorbereiding

5.3 Gebruikte tools en methodologie

- GitHub — versiebeheer en samenwerking (repository met branchprotectie)
- Trello — Kanban-bord voor taakopvolging (To Do / In Progress / Done)
- Postman — API testen en simulatie van de volledige betalingscyclus
- Docker Desktop — lokale containerisatie tijdens ontwikkeling
- Visual Studio Code — code-editor
- Odisee Toledo — communicatie en uploads

5.4 Problemen en oplossingen

Tijdens het project kwamen we volgende technische problemen tegen en losten we ze als volgt op:

Probleem	Oplossing
CB API gaf wisselende tokenformaten terug (soms token direct, soms in data-object)	cbClient.js aangepast: resp.data?.token || resp.data?.data?.token

Probleem	Oplossing
MySQL DATETIME-kolommen kwamen als Date-objecten terug in plaats van strings	mysql2 pool configuratie: dateStrings: true toegevoegd
Docker-container startte op voor MariaDB klaar was	Dockerfile aangepast: nc-loop om op DB_HOST:DB_PORT te wachten voor de app start
Dubbele ACKs werden verwerkt na herstart van de poller	INSERT IGNORE toegevoegd op ack_in en po_in tabellen
Rekeningsaldo kon negatief worden door race conditions	MySQL-transacties (BEGIN/COMMIT/ROLLBACK) gebruikt bij alle saldo-aanpassingen
docker-compose.yml mocht niet in de GitHub repo staan (network team vereiste)	docker-compose.yml toegevoegd aan .gitignore, .env.example aangemaakt met alle variabelen

5.5 Cross-team testresultaten

Op woensdag 29 april werden de APIs getest met andere teams via de centrale Clearing Bank (IP: 192.168.137.251:5000). De volgende tabel toont de testresultaten:

Test	Resultaat
`GET /api/info — eigen endpoint`	Geslaagd: BIC, naam en teamleden correct teruggegeven
`POST betaling naar ander team (OB-rol)`	Geslaagd
Ontvangen betaling van ander team (BB-rol)	Geslaagd
Foutscenario: bedrag > €500	Geslaagd: code 4002 correct teruggegeven door CB
Foutscenario: onbekende BB	Geslaagd: code 4004 correct teruggegeven door CB
Docker deployment op centrale server	Afhankelijk van Netwerk Team

6. Handleiding voor het gebruik van de GUI

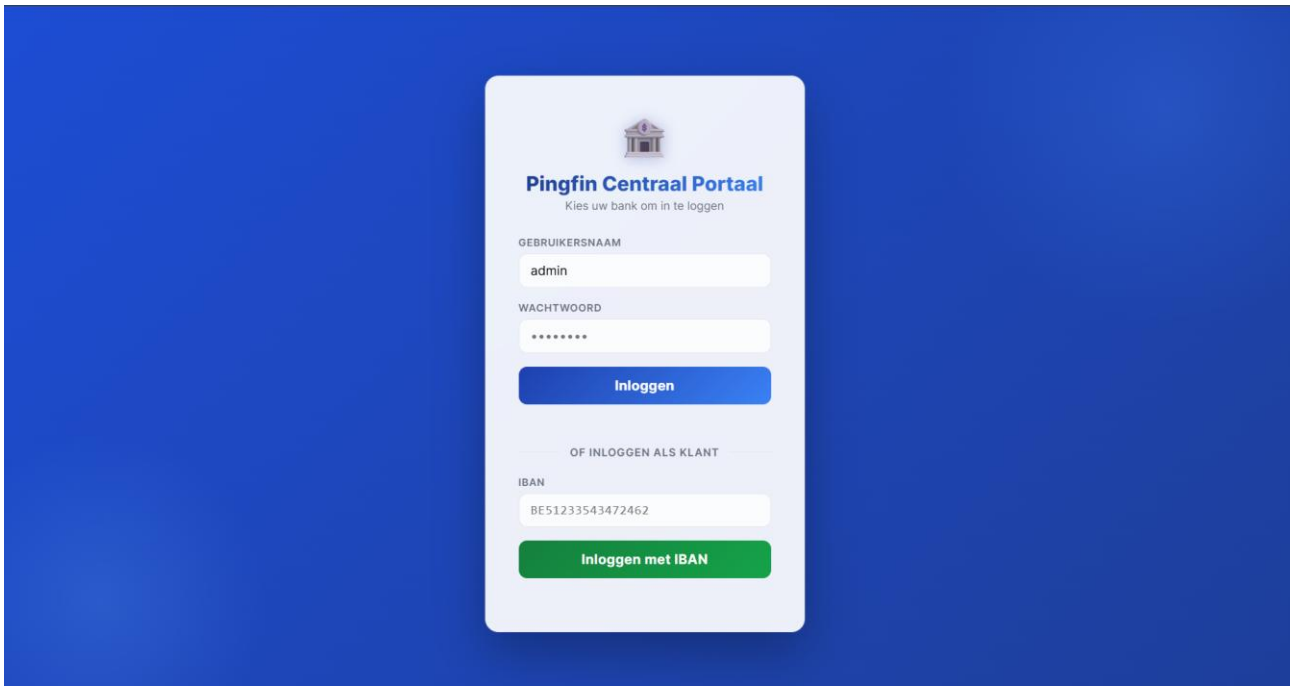
6.1 Overzicht

De GUI is een single-page applicatie (public/index.html) die via de browser toegankelijk is op http://<server-ip>:3000. Er zijn twee gebruikersrollen met elk een eigen interface.

Rol	Toegang
admin	Volledige toegang: dashboard, accounts beheren, betalingen initiëren, PO-monitor, logs, banks, gebruikersbeheer
user	Beperkte toegang: eigen rekening bekijken, betaling doen, betalingshistorie raadplegen

6.2 Inloggen

Navigeer naar http://<server-ip>:3000. Vul gebruikersnaam en wachtwoord in op het loginscherm. Standaard credentials: admin / admin123. Na succesvolle login wordt de interface automatisch geladen op basis van de rol. Het JWT-token wordt in localStorage bewaard en is 4 uur geldig.



6.3 Admin-interface — functionaliteiten

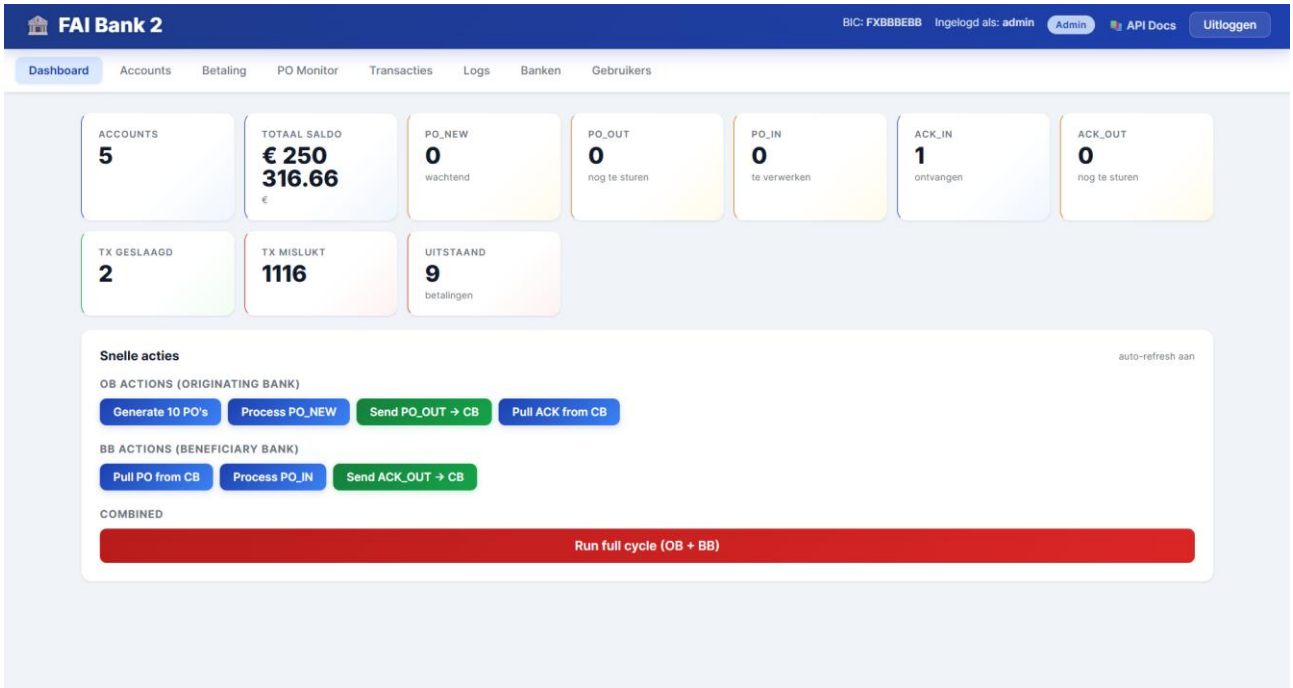
De admin-interface bestaat uit 8 tabs die elk een ander aspect van de bankoperatie beheren:

Dashboard

Het dashboard toont live statistieken die elke 5 seconden automatisch worden vernieuwd:

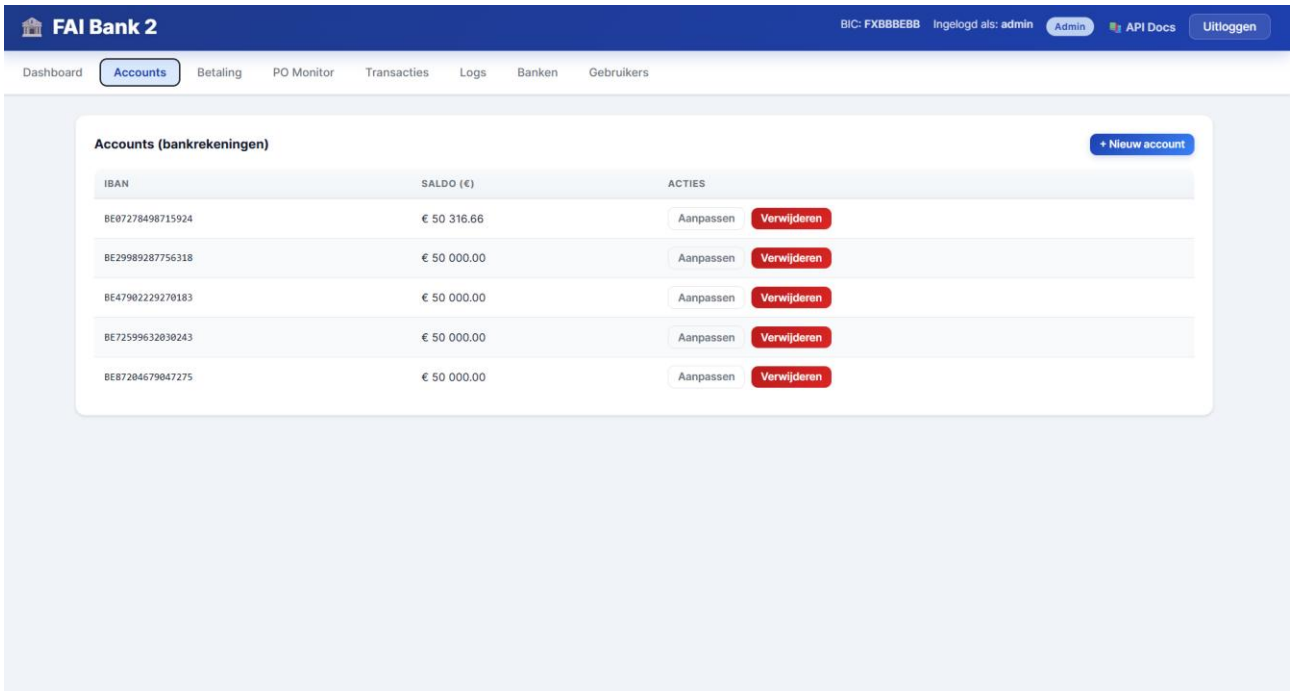
- Aantal accounts en totaal saldo
- Tellingen per PO-tabel (PO_NEW, PO_OUT, PO_IN, ACK_IN, ACK_OUT)
- Geslaagde en mislukte transacties
- Aantal uitstaande betalingen (PO_OUT verzonden maar nog geen ACK ontvangen)

Quick action-knoppen: "Generate & run cycle" (genereert 10 POs en doorloopt de volledige cyclus), "Run cycle" (voert de cyclus uit zonder nieuwe POs), "Pull POs from CB" en "Pull ACKs from CB".



Accounts

Overzicht van alle bankrekeningen met IBAN en saldo. De admin kan nieuwe rekeningen aanmaken via een modal (IBAN + beginsaldo), saldo's aanpassen, en rekeningen verwijderen (enkel als saldo = 0 en geen openstaande POs).

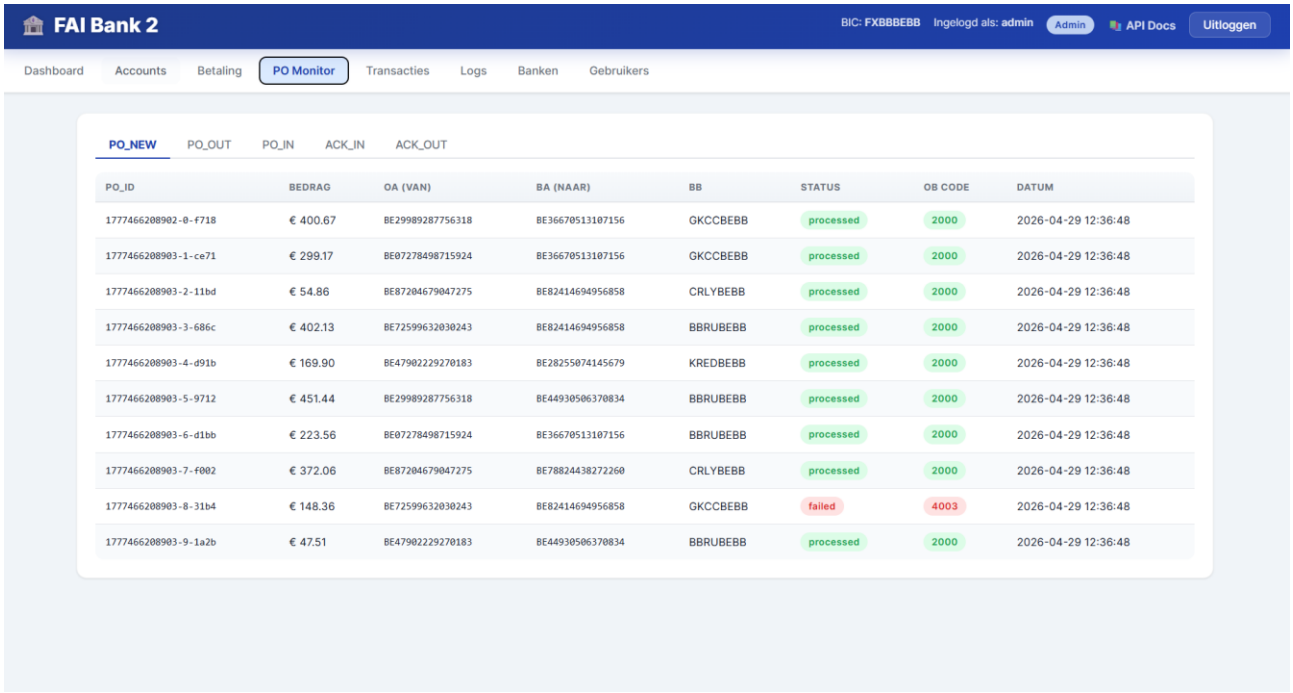


Betalingen

Formulier om een betaling te starten: kies een bronrekening (dropdown), vul doel-IBAN en doel-BIC in, bedrag (max €500) en betalingsbericht. De API valideert alle velden en stuurt de betaling direct naar de CB. Het resultaat (geslaagd / foutcode) wordt getoond.

PO Monitor

Tabbladen voor PO_NEW, PO_OUT, PO_IN, ACK_IN en ACK_OUT. Elke tabel toont de 200 meest recente records, gesorteerd op datum (nieuwste eerst). Status-badges geven in één oogopslag de verwerkingsstatus weer (pending, processed, failed).



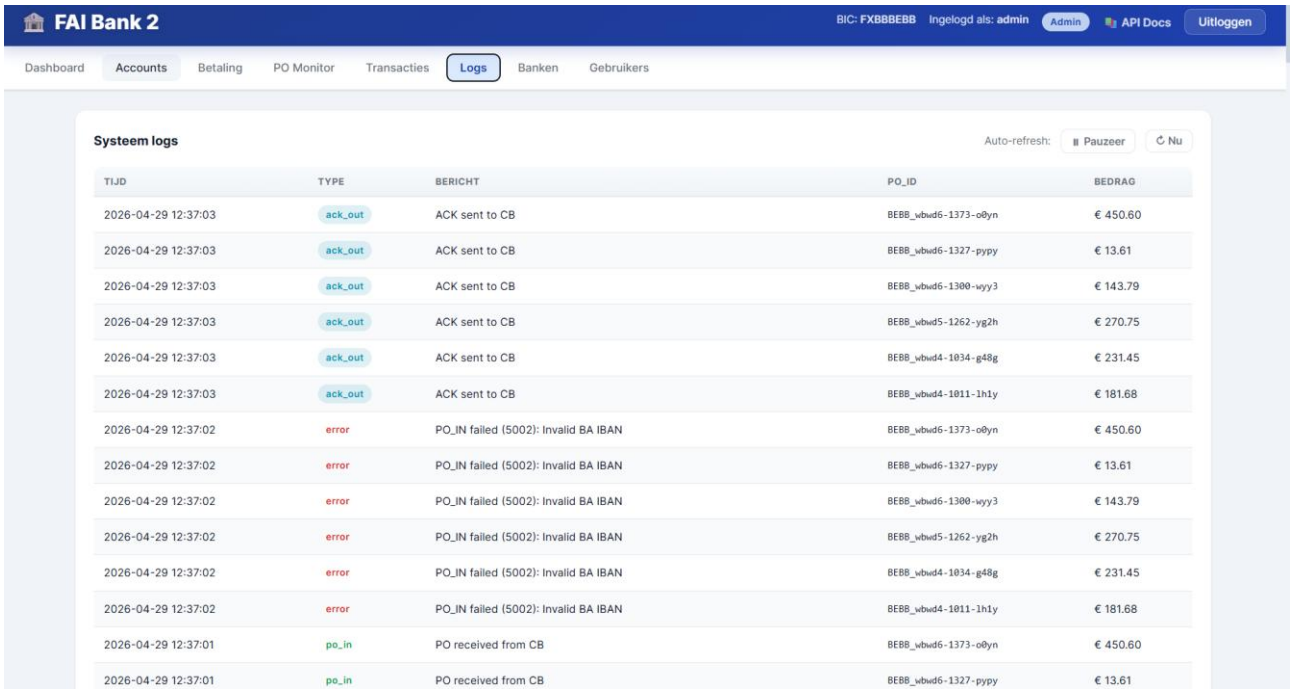
PO_ID	BEDRAG	OA (VAN)	BA (NAAR)	BB	STATUS	OB CODE	DATUM
1777466208902-0-f718	€ 400.67	BE29989287756318	BE36670513107156	GKCCBEBB	processed	2000	2026-04-29 12:36:48
1777466208903-1-ce71	€ 299.17	BE07278498715924	BE36670513107156	GKCCBEBB	processed	2000	2026-04-29 12:36:48
1777466208903-2-11bd	€ 54.86	BE87204679047275	BE82414694956858	CRLYBEBB	processed	2000	2026-04-29 12:36:48
1777466208903-3-686c	€ 402.13	BE72599632030243	BE82414694956858	BBRUBEBB	processed	2000	2026-04-29 12:36:48
1777466208903-4-d91b	€ 169.90	BE47902229270183	BE28255074145679	KREDBEBB	processed	2000	2026-04-29 12:36:48
1777466208903-5-9712	€ 451.44	BE29989287756318	BE44930506370834	BBRUBEBB	processed	2000	2026-04-29 12:36:48
1777466208903-6-d1bb	€ 223.56	BE07278498715924	BE36670513107156	BBRUBEBB	processed	2000	2026-04-29 12:36:48
1777466208903-7-f002	€ 372.06	BE87204679047275	BE78824438272260	CRLYBEBB	processed	2000	2026-04-29 12:36:48
1777466208903-8-31b4	€ 148.36	BE72599632030243	BE82414694956858	GKCCBEBB	failed	4003	2026-04-29 12:36:48
1777466208903-9-1a2b	€ 47.51	BE47902229270183	BE44930506370834	BBRUBEBB	processed	2000	2026-04-29 12:36:48

Transacties

Overzicht van alle rekeningtransacties. Filterbaar per rekening via een dropdown.

Logs

Live feed van alle systeemgebeurtenissen. Events worden kleurgecodeerd op type: fout (rood), po_in/po_out (blauw), ack_in/ack_out (groen), general (grijs). De feed ververst automatisch.



TIJD	TYPE	BERICHT	PO_ID	BEDRAG
2026-04-29 12:37:03	ack_out	ACK sent to CB	BEBB_wbwd6-1373-o0yn	€ 450.60
2026-04-29 12:37:03	ack_out	ACK sent to CB	BEBB_wbwd6-1327-pypy	€ 13.61
2026-04-29 12:37:03	ack_out	ACK sent to CB	BEBB_wbwd6-1300-wyy3	€ 143.79
2026-04-29 12:37:03	ack_out	ACK sent to CB	BEBB_wbwd5-1262-yg2h	€ 270.75
2026-04-29 12:37:03	ack_out	ACK sent to CB	BEBB_wbwd4-1034-g48g	€ 231.45
2026-04-29 12:37:03	ack_out	ACK sent to CB	BEBB_wbwd4-1011-1h1y	€ 181.68
2026-04-29 12:37:02	error	PO_IN failed (5002): Invalid BA IBAN	BEBB_wbwd6-1373-o0yn	€ 450.60
2026-04-29 12:37:02	error	PO_IN failed (5002): Invalid BA IBAN	BEBB_wbwd6-1327-pypy	€ 13.61
2026-04-29 12:37:02	error	PO_IN failed (5002): Invalid BA IBAN	BEBB_wbwd6-1300-wyy3	€ 143.79
2026-04-29 12:37:02	error	PO_IN failed (5002): Invalid BA IBAN	BEBB_wbwd5-1262-yg2h	€ 270.75
2026-04-29 12:37:02	error	PO_IN failed (5002): Invalid BA IBAN	BEBB_wbwd4-1034-g48g	€ 231.45
2026-04-29 12:37:02	error	PO_IN failed (5002): Invalid BA IBAN	BEBB_wbwd4-1011-1h1y	€ 181.68
2026-04-29 12:37:01	po_in	PO received from CB	BEBB_wbwd6-1373-o0yn	€ 450.60
2026-04-29 12:37:01	po_in	PO received from CB	BEBB_wbwd6-1327-pypy	€ 13.61

Banks

Lijst van alle deelnemende banken, rechtstreeks opgehaald van de Clearing Bank via GET /api/banks.

Gebruikers

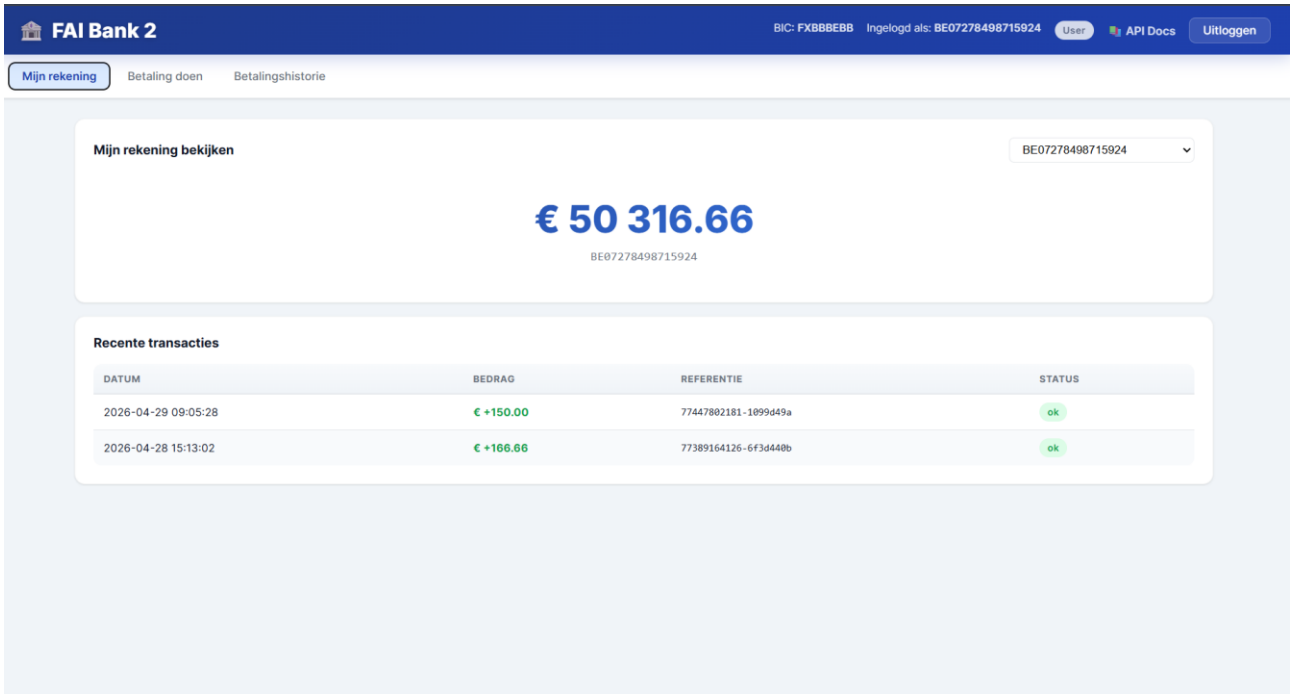
Overzicht van alle GUI-gebruikers. Nieuwe gebruikers aanmaken (username, wachtwoord, rol). Gebruikers verwijderen (niet de eigen account).

6.4 User-interface — functionaliteiten

De user-interface is vereenvoudigd en toont enkel de functies die relevant zijn voor een gewone rekeninghouder:

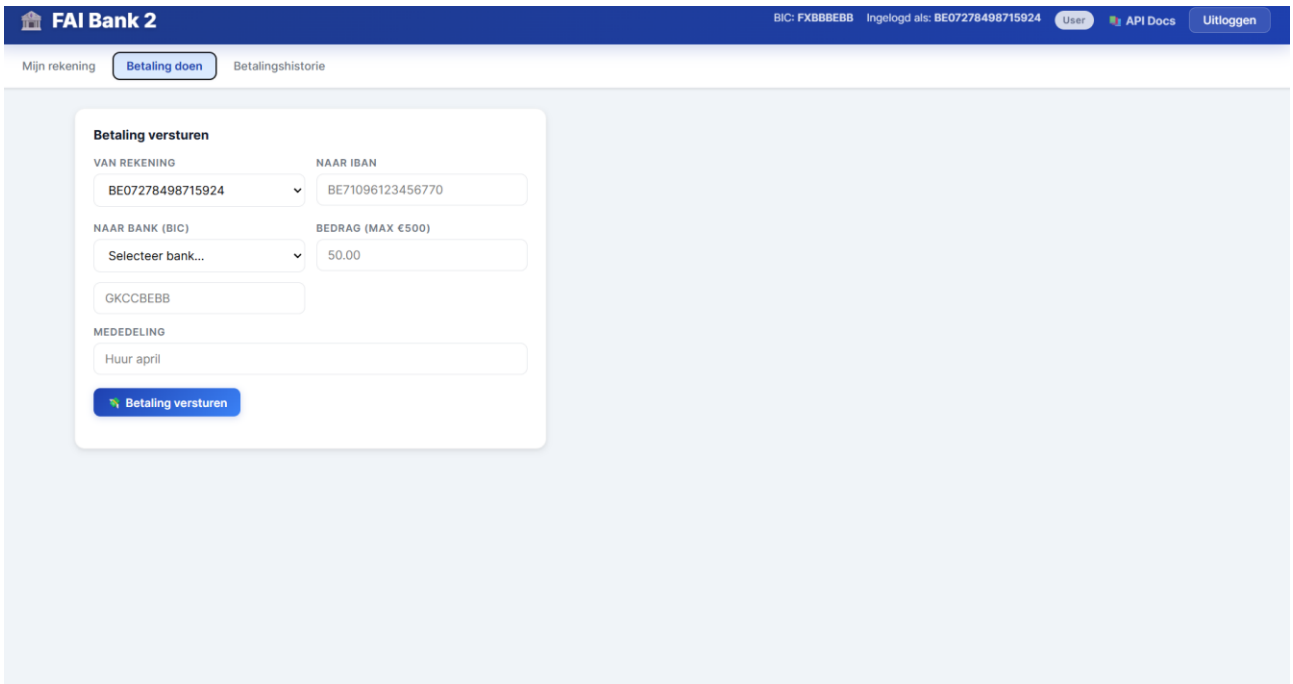
Mijn rekening

Selecteer een rekening via de dropdown. De pagina toont het huidige saldo en een tabel van recente transacties voor die rekening.



Betaling doen

Formulier met: bronrekening (dropdown eigen IBANs), bestemmings-IBAN (met real-time IBAN-validatiefeedback), bestemmings-BIC (dropdown bekende banken of vrij invoer), bedrag (max €500 met validatie) en betalingsbericht. Na klikken op "Verzenden" wordt de betaling verstuurd en het resultaat direct getoond.



Betalingshistorie

Tabel van alle POs waarbij de eigen rekeningen betrokken zijn, gesorteerd op datum.

7. Reflectie

7.1 Terugblik op het project

Het Pingfin-project was een uitdagende maar leerrijke ervaring. In vier dagen hebben we een volledig werkende banktoepassing gebouwd die communiceert met andere teams via een gestandaardiseerde API. De combinatie van nieuwe technologieën (Node.js, Express.js, Docker) met een complex berichtenprotocol (SEPA-flow) zorgde voor een steile leercurve, maar ook voor een groot gevoel van voldoening wanneer de eerste cross-team betalingen succesvol verliepen.

De samenwerking binnen het team verliep vlot. Dagelijkse stand-ups hielden iedereen op de hoogte van de voortgang. Het Trello-bord hielp om taken te verdelen en de planning bij te houden. GitHub zorgde voor een gestructureerde werkwijze met branches en pull requests.

7.2 Geleerde lessen

- Docker Compose maakt deployment aanzienlijk eenvoudiger, maar vereist goede aandacht voor de opstartvolgorde van services (health checks).
- Het testen van API-endpoints via Postman, vóór de GUI gebouwd is, bespaart veel tijd bij het debuggen.
- Duidelijke foutcodes en een consistent responsformaat (ok/status/code/message/data) maken het debuggen van communicatieproblemen veel sneller.
- MySQL-transacties zijn essentieel bij gelijktijdige saldo-aanpassingen — zonder transacties kunnen race conditions optreden.
- De background poller automatiseert de betalingscyclus, maar vraagt om goede idempotentie (INSERT IGNORE, dubbele ACK-check) om problemen bij herstarts te voorkomen.
- Communicatie met andere teams over hun BIC-codes en eindpunten is cruciaal voor cross-team testen. Dit had eerder gepland kunnen worden.

7.3 Verbeterpunten voor toekomstige projecten

- Eerder beginnen met het plannen van cross-team communicatie — wie heeft welke BIC, welk IP, welke poort.
- Een retry-mechanisme implementeren voor mislukte CB-requests (momenteel herprobeert de poller, maar zonder exponentiële backoff).
- Unit tests schrijven voor de validators en processors — dit had veel debugtijd bespaard.
- Swagger/OpenAPI-documentatie genereren vanuit de code, in plaats van handmatig te schrijven.
- De GUI uitbreiden met een real-time notificatiesysteem (WebSockets) zodat inkomende betalingen direct zichtbaar zijn zonder pagina te verversen.
- Een dashboard voor uitstaande betalingen met automatische timeout-melding na 1 uur, conform de businessregel uit de manual.

8. Bijlagen

Bijlage A — Databankschema (DDL)

Vereenvoudigd overzicht van de kernstructuur (volledig schema in src/db/schema.sql):

```
CREATE TABLE accounts (
  id      VARCHAR(34) PRIMARY KEY,  -- IBAN
  balance DECIMAL(10,2) NOT NULL CHECK (balance >= 0)
);
```

```
CREATE TABLE po_out (
  po_id       VARCHAR(50) PRIMARY KEY,
  po_amount   DECIMAL(10,2),
  po_message  VARCHAR(255),
  po_datetime DATETIME,
  ob_id       VARCHAR(20), oa_id VARCHAR(34),
  ob_code     VARCHAR(10), ob_datetime DATETIME,
  cb_code     VARCHAR(10), cb_datetime DATETIME,
  bb_id       VARCHAR(20), ba_id VARCHAR(34),
  bb_code     VARCHAR(10), bb_datetime DATETIME,
  sent_to_cb  TINYINT(1) DEFAULT 0
);
```

Bijlage B — Docker Compose configuratie

De docker-compose.yml die gebruikt wordt voor deployment op de centrale server (conform network team guidelines):

```
services:
  db:
    image: mysql:8
    environment:
      MYSQL_DATABASE: ${DB_NAME}
      MYSQL_USER: ${DB_USER}
      MYSQL_PASSWORD: ${DB_PASSWORD}
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
    volumes:
      - db_data:/var/lib/mysql
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      retries: 5
    networks:
      - pingfin-net
  api:
    build: .
    ports:
      - "${PORT}:3000"
    environment:
      CB_BASE_URL: ${CB_URL:-https://stevenop.be/pingfin/api/v2}
    depends_on:
```

```
      db:
        condition: service_healthy
    networks:
      - pingfin-net
networks:
  pingfin-net:
    external: true
    name: pingfin-net
```

Bijlage C — Validatieregels overzicht

Alle validatieregels die door de API gecontroleerd worden bij het verwerken van Payment Orders:

Veld	Regel
po_id	Max 50 tekens, moet beginnen met {OB_BIC}_ (bv. BOFABE3X_...)
po_amount	Eindige waarde > 0, maximaal 2 decimalen, maximaal €500,00
po_datetime	Formaat YYYY-MM-DD HH:MM:SS (bv. 2026-04-28 10:30:45)
ob_id / bb_id	BIC: 8 of 11 tekens, regex /^[A-Z]{4}[A-Z]{2}[A-Z0-9]{2}([A-Z0-9]{3})?$/
oa_id / ba_id	IBAN: mod-97 checksum, geen spaties, 15–34 alfanumerieke tekens
Saldo	Mag nooit negatief worden (CHECK-constraint + transactielock)
OB ≠ BB	Interne betalingen (OB == BB) worden geweigerd met code 3007

Bijlage D — Clearing Bank endpoints

De externe Clearing Bank API (stevenop.be tijdens ontwikkeling, interne server bij deployment) biedt de volgende endpoints:

Methode	Endpoint	Beschrijving
`POST`	`/token`	Token aanvragen (body: { bic, secret_key })
`GET`	`/banks`	Lijst van deelnemende banken
`POST`	`/banks`	Bankinformatie bijwerken (naam + teamleden)
`POST`	`/po_in`	POs insturen als OB
`GET`	`/po_out`	POs ophalen als BB (/po_out/test/true voor testmodus)
`POST`	`/ack_in`	ACKs insturen als BB
`GET`	`/ack_out`	ACKs ophalen als OB (/ack_out/test/true voor testmodus)
`GET`	`/errorcodes`	Foutcodes opvragen (geen authenticatie vereist)

Bijlage E — Postman Testing

Accounts Bank 1

GET http://localhost:3000/api/accounts/

Authorization: Bearer Token

200 OK · 38 ms · 1.26 KB


```
{
    "ok": true,
    "status": 200,
    "code": "2000",
    "message": "Lijst van alle accounts succesvol opgehaald",
    "data": [
        {
            "id": "BE16473829104756",
            "balance": 5000
        },
        {
            "id": "BE19384756283910",
            "balance": 5000
        },
        {
            "id": "BE21736485019274",
            "balance": 5000
        },
        {
            "id": "BE28947265183047",
            "balance": 5000
        },
        {
            "id": "BE37192847563019",
            "balance": 5000
        },
        {
```


Accounts - Bank1 (Success)

GET http://localhost:3000/api/accounts/

200 OK · 13 ms · 1.26 KB


```
{
    "ok": true,
    "status": 200,
    "code": "2000",
    "message": "Lijst van alle accounts succesvol opgehaald",
    "data": [
        {
            "id": "BE16473829104756",
            "balance": 4850
        },
        {
            "id": "BE19384756283910",
            "balance": 5000
        },
        {
            "id": "BE21736485019274",
            "balance": 5000
        },
        {
            "id": "BE28947265183047",
            "balance": 5000
        },
        {
            "id": "BE37192847563019",
            "balance": 5000
        },
        {
            "id": "BE38291746503928",
            "balance": 5000
        },
        {
            "id": "BE45637281947563",
```

Accounts Bank 2

GET http://localhost:3001/api/accounts/

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (12) Test Results — 200 OK · 31 ms · 1.26 KB


```
{
    "ok": true,
    "status": 200,
    "code": "2000",
    "message": "Lijst van alle accounts succesvol opgehaald",
    "data": [
        {
            "id": "BE16473829104756",
            "balance": 5000
        },
        {
            "id": "BE19384756283910",
            "balance": 5000
        },
        {
            "id": "BE21736485019274",
            "balance": 5000
        },
        {
            "id": "BE28947265183047",
            "balance": 5000
        },
        {
            "id": "BE37192847563019",
            "balance": 5000
        },
```


Bank 2 - Accounts (Success)

GET http://localhost:3001/api/accounts/

Body Cookies Headers (12) Test Results — 200 OK · 9 ms · 1.26 KB


```
{
    "ok": true,
    "status": 200,
    "code": "2000",
    "message": "Lijst van alle accounts succesvol opgehaald",
    "data": [
        {
            "id": "BE16473829104756",
            "balance": 5000
        },
        {
            "id": "BE19384756283910",
            "balance": 5150
        },
        {
            "id": "BE21736485019274",
            "balance": 5000
        },
        {
            "id": "BE28947265183047",
            "balance": 5000
        },
        {
            "id": "BE37192847563019",
            "balance": 5000
        },
        {
            "id": "BE38291746503928",
            "balance": 5000
        },
        {
            "id": "BE45637281947563",
```

Bank 2 - PO verwerkt

GET http://localhost:3001/api/po_in_process/

Authorization: Bearer Token

Response 200 OK, 17 ms, 424 B

```
{
    "ok": true,
    "message": "2 inkomende PO's succesvol verwerkt."
}
```

Test Bank 1 => Bank 2

POST http://localhost:3000/api/po_new_add/

Body (raw, JSON)

```
{
  "data": [
    {
      "po_id": "B0FABE3X_TEST002",
      "po_amount": 150.00,
      "po_message": "Eerste echte test naar Bank 2!",
      "po_datetime": "2026-04-28 12:00:00",
      "ob_id": "B0FABE3X",
      "oa_id": "BE16473829104756",
      "bb_id": "FXBBBEBB",
      "ba_id": "BE19384756283910"
    }
  ]
}
```

Response 200 OK, 15 ms, 478 B

```
{
    "ok": true,
    "status": 200,
    "code": "2000",
    "message": "OK",
    "data": {
        "inserted_count": 1,
        "inserted": [
            "B0FABE3X_TEST002"
        ]
    }
}
```

Bank 1 - PO validatie

GET http://localhost:3000/api/po_new_process/

Authorization — Auth Type: Bearer Token — Token: ••••••••••••••••••••••••••••••••

The authorization header will be automatically generated when you send the request. Learn more about Bearer Token authorization.

Body — 200 OK · 13 ms · 454 B

```
{
    "ok": true,
    "status": 200,
    "code": "2000",
    "message": "Succes: 1 PO's gevalideerd en verwerkt."
}
```

Bank 2 - Pull

GET http://localhost:3001/api/po_pull/

Authorization — Auth Type: Bearer Token — Token: ••••••••••••••••••••••••••••••••

The authorization header will be automatically generated when you send the request. Learn more about Bearer Token authorization.

Body — 200 OK · 122 ms · 416 B

```
{
    "ok": true,
    "message": "2 nieuwe PO's binnengehaald."
}
```

Bank 1 - PO Verzonden

GET http://localhost:3000/api/po_out_send/

Authorization: Bearer Token

Token: ••••••••••••••••••••••••••••••••

The authorization header will be automatically generated when you send the request. Learn more about Bearer Token authorization.

200 OK · 203 ms · 517 B


```
{
    "ok": true,
    "status": 200,
    "code": "2000",
    "message": "1 PO's verzonden!",
    "cb_response": {
        "status": 200,
        "data": "Result: 1 Payment Orders Processed.",
        "ok": true
    }
}
```


Token Bank 1

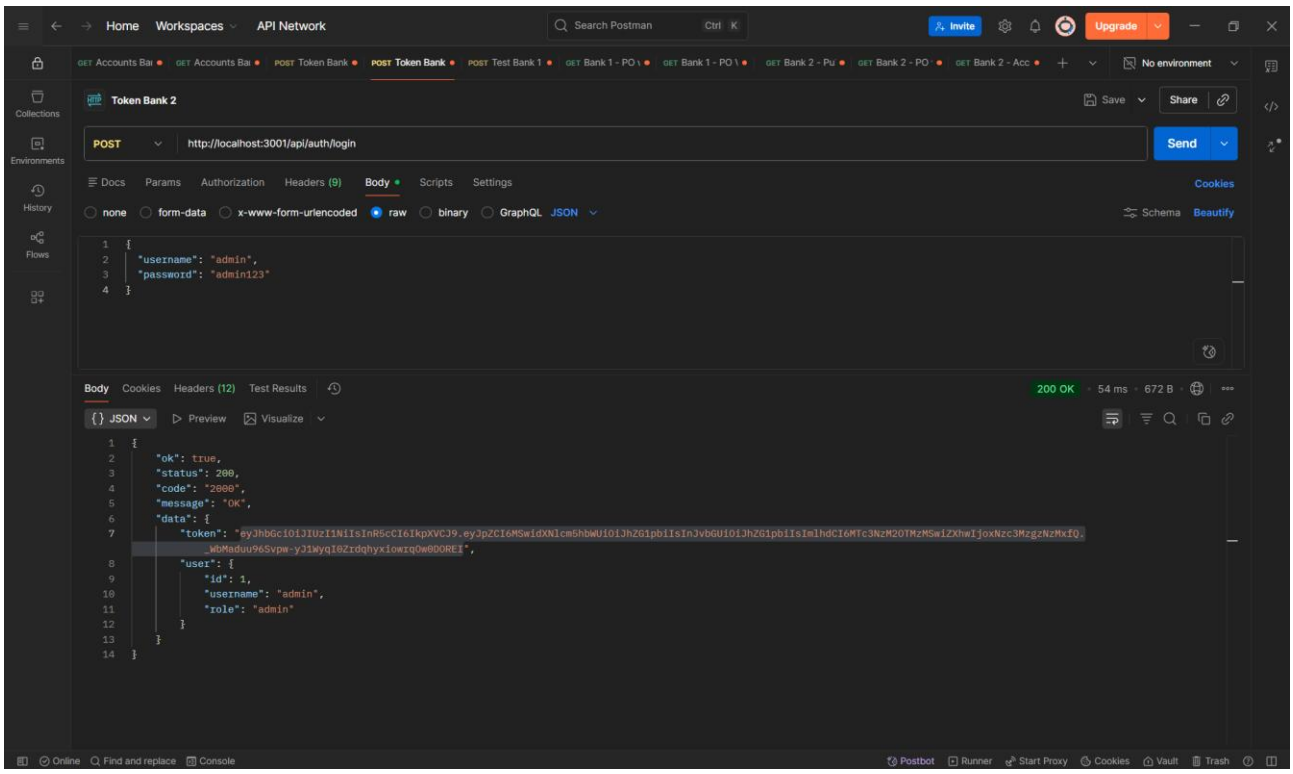
POST http://localhost:3000/api/auth/login

Body: raw, JSON

```
{
  "username": "admin",
  "password": "admin123"
}
```

200 OK · 62 ms · 672 B


```
{
    "ok": true,
    "status": 200,
    "code": "2000",
    "message": "OK",
    "data": {
        "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidXNlcm5hbWUiOiJhZG1pbiIsInJvbGUiOiJhZG1pbiIsImlhdCI6MTc3NzM2OTI2OCwiZXhwIjoxNzc3MzgzNjY4fQ.
            _uHSxI-UO77FlSZs5Lc5C_C489ohcLM3arYQrtg7bBI",
        "user": {
            "id": 1,
            "username": "admin",
            "role": "admin"
        }
    }
}
```


Home
Workspaces
API Network
Search Postman
Invite
Upgrade
Collections
Environments
History
Flows
Token Bank 2
Save
Share
POST
http://localhost:3001/api/auth/login
Send
Docs
Params
Authorization
Headers (9)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"username": "admin",
"password": "admin123"
}
Body
Cookies
Headers (12)
Test Results
200 OK
54 ms
672 B
JSON
Preview
Visualize
{
"ok": true,
"status": 200,
"code": "2000",
"message": "OK",
"data": {
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidXNlcm5hbWUiOiJhZG1pbiIsInJvbGUiOiJhZG1pbiIsImlhdCI6MTc3NzM2OTMzMSwiZXhwIjoxNzc3MzgzNzMxfQ.
_WbMaduu96Svpw-yJ1WyqI0ZrdqhyxiowrqOw0DOREI",
"user": {
"id": 1,
"username": "admin",
"role": "admin"
}
}
}
Online
Find and replace
Console
Postbot
Runner
Start Proxy
Cookies
Vault
Trash